EngiSync — Project Walkthrough

Improvements, Challenges & Fixes, and Functional Roadmap

Prepared for Tafadzwa Musendo

26 July 2026

Contents

1. Introduction

EngiSync is a production-grade web platform built for university engineering students to manage individual work, group projects, collaboration, resource sharing, meetings, budgets, and AI-assisted engineering support. This document is a complete walk-through of the work carried out on the platform: what was built and improved, the problems that came up along the way and how each was diagnosed and fixed, and a functional roadmap showing how the pieces of the system connect.

Stack: Next.js 15 (App Router, Server Components, Server Actions, Route Handlers doubling as the backend), Prisma 6 ORM on PostgreSQL (Neon), Auth.js v5 (Credentials + Google + Microsoft Entra ID + TOTP 2FA), Tailwind CSS, a provider-agnostic AI layer (Gemini by default), and a provider-agnostic email layer (Resend or SMTP).

---

2. Improvements Delivered

2.1 Analytics, Reporting & Documentation (Phases 4–5)

- **Lecturer/Supervisor Analytics** — per-student contribution percentage, task completion, attendance, productivity, file activity, and comment counts, exportable as PDF reports.
- **Full documentation set** — Software Requirements Specification (SRS), Development Journal, Configuration Guide, User Manual, and Maintenance Manual, giving the project the paper trail expected of a production system.

2.2 AI Assistant Hardening

- Fixed a broken AI integration (deprecated Gemini model) by moving to `gemini-2.5-flash`, with automatic retry on rate-limit responses and clearer error messages.
- Removed a hidden bug where the Resource Hub called the AI on every page load, silently burning free-tier quota; recommendations now default to a fast heuristic and only call AI when a user explicitly asks.
- Added a **master kill switch** (`AI_DISABLED`). At the user's request, all AI features were disabled and the API key removed from the environment; any AI action now cleanly returns "out of service" instead of failing or exposing configuration.

2.3 Mobile Experience

- **Root-caused and fixed** a broken mobile navigation menu: the header's blur effect was trapping the slide-out drawer inside a 64px strip instead of the full screen. Fixed by rendering the drawer through a React portal straight onto the page body.
- Added a **bottom navigation bar** for phones (Home / Tasks / Groups / Calendar), sized for touch and respecting device safe areas.
- Converted wide data tables to **stacked card layouts** on small screens across the supervisor report and other data-heavy pages.
- Rebuilt the **calendar** for mobile: the desktop month grid is replaced with a scrollable agenda list grouped by day.
- Audited every dashboard grid and stat block for responsive breakpoints; fixed the one genuine gap found (the calendar) and confirmed the rest were already correctly responsive.

2.4 Email & Verification

- Diagnosed and fixed **completely silent email verification** — no provider was configured, and three separate code paths were swallowing the resulting errors instead of surfacing them. Email sending now returns a real success/failure result, failures are logged, and users see an honest message if a resend fails.

2.5 Security & Privacy Hardening

A full audit was carried out covering authentication, authorization, injection, XSS/CSRF, SSRF, file upload, AI abuse, and rate limiting. Concrete fixes shipped:

- **PII exposure fixed** — raw email addresses were being rendered in public-facing UI (department pages, workspace member lists, meetings, task assignments). Replaced everywhere with a single safe `displayName()` helper so an email is never the fallback shown to other users; only the account owner or an admin can see it.
- **Security headers** — added a full Content-Security-Policy, HTTP Strict-Transport-Security (2-year, preload), disabled DNS prefetching, and set `no-store` caching on all API responses.
- **OAuth login bug fixed** — Google sign-in was failing with `invalid_client`. Root cause was two compounding bugs: the code read one set of environment variable names while the setup docs used another, and the Google/Microsoft buttons were shown even when not configured. Both are now handled — variable names are matched flexibly, and buttons only appear when a provider is actually usable.
- **Account deletion & identity lifecycle** — built a complete, safe account-deletion flow:
  - Requires password re-entry (and 2FA code, if enabled) plus an emailed confirmation link before anything happens.
  - Blocked if the user is the sole leader of an active group — they must transfer leadership or archive the project first.
  - Shared academic records (feedback, report versions, resource contributions) are anonymized to “Deleted user” rather than destroyed, preserving group history.
  - Purely personal data (sessions, tokens, learner profile) is hard-deleted.
  - The deleted account’s email is placed in a 30-day cooling-off reservation so it cannot be immediately reused to re-register or impersonate the former user.

2.6 Navigation & Information Architecture Redesign

User feedback flagged “too many visible buttons” and navigation fatigue. Without removing a single feature, the interface was reorganized:

- **Sidebar consolidated from 7 sections to 4** — *Home*, *My work*, *Learning*, *Insights*, *Account* — grouped by what the user is trying to do rather than by internal module.
- **Global command palette (⌘K / Ctrl+K)** — a search box in the header that jumps straight to any page by typing what you want (e.g. “meeting”, “budget”), with full keyboard navigation. This is what makes the lighter sidebar safe: nothing is hidden, it’s just not all on screen at once.
- **Progressive disclosure on busy pages** — the workspace page’s five equal-weight buttons were reduced to three primary actions (Tasks, Documentation, Discussions) with the remaining actions (Quizzes, AI Evaluation, Version History) moved into a “More” menu.
- Kept the existing state-aware “Get set up” onboarding checklist rather than adding a redundant guided tour.

2.7 Login Page Redesign

Rebuilt the login page to a modern two-panel layout: an animated brand panel alongside a form panel with floating-label inputs, inline validation, a password visibility toggle, social sign-in buttons that only appear when actually configured, and full accessibility support (keyboard focus, screen-reader live regions, reduced-motion handling).

---

3. Challenges, Root Causes & Fixes

The table below is a record of every non-trivial problem hit during this build, how it was actually diagnosed (not guessed), and what fixed it.

#	Problem	Root Cause	Fix
1	AI Assistant returned a 404 error	The configured Gemini model name had been deprecated by Google	Updated to the current `gemini-2.5-flash` model
2	AI Assistant hit 429 rate-limit errors	Free-tier quota exhausted, partly due to a hidden auto-call on every Resource Hub page load	Added retry-with-backoff, and made the recommendation feature opt-in instead of automatic
3	Mobile hamburger menu “did something” but never opened properly	The navbar’s blur effect created a CSS containing block that trapped the fixed-position drawer inside the header strip	Rendered the drawer through a React portal directly onto the page body
4	Email verification never arrived, no error shown	No email provider was configured, and three separate code paths silently swallowed the failure	Email sending now returns a real result object; failures are logged and surfaced to the user instead of hidden

#	Problem	Root Cause	Fix
5	Build failed on `npm run build`	An unescaped apostrophe in JSX text tripped a lint rule	Replaced with the proper HTML entity; grepped the rest of the new code for the same pattern
6	Google sign-in failed with `invalid_client`	Two compounding bugs: environment variable naming mismatch between code and docs, and OAuth buttons rendered even when unconfigured	Code now accepts either variable naming convention, and buttons/providers are only enabled when real credentials are present
7	Prisma commands failed in the build sandbox	The sandbox cannot reach Prisma’s binary CDN	Established a workaround: scoped TypeScript checks on non-Prisma code in-sandbox, full verification via the user’s own `npm run build`
8	Email addresses were visible to other users in group/department views	No single safe accessor for identity — every page independently decided whether to show name or email	Introduced one `displayName()` helper as the only sanctioned way to render identity anywhere in the app

---

4. Functional Roadmap — How Everything Connects

```
                         ┌─────────────────────────┐
                         │          Guest          │
                         │  (marketing + login)    │
                         └────────────┬────────────┘
                                      │ register / sign in
                                      ▼
                 ┌───────────────────────────────────────┐
                 │             Authentication            │
                 │  Email+password · Google · Microsoft  │
                 │  Email verification · 2FA (TOTP)      │
                 └───────────────────┬───────────────────┘
                                     ▼
                 ┌───────────────────────────────────────┐
                 │               Dashboard               │
                 │  Command palette · Notifications      │
                 │  Tasks · Deadlines · Calendar · Stats │
                 └───┬───────────┬──────────┬────────────┘
                     ▼           ▼          ▼
         ┌────────────────┐ ┌─────────────┐ ┌────────────────┐
         │ Group          │ │ Personal    │ │ Departments &  │
         │ Workspaces     │ │ Projects    │ │ Resource Hub   │
         │ (Leader/Member)│ │             │ │                │
         └───────┬────────┘ └─────┬───────┘ └───────┬────────┘
                 │                │                 │
  ┌──────────────┼────────┬───────┴────────┐        │
  ▼              ▼        ▼                ▼        ▼
┌──────────┐ ┌────────────┐ ┌─────────┐ ┌───────────┐ ┌──────────┐
│ Tasks &  │ │ Meetings & │ │ Budget  │ │ Documents │ │ AI       │
│Milestones│ │ Attendance │ │ (EcoCash│ │ & Reports │ │ Assistant│
│          │ │            │ │tracking)│ │           │ │ (gated)  │
└────┬─────┘ └─────┬──────┘ └────┬────┘ └─────┬─────┘ └────┬─────┘
     │             │             │            │            │
     └───────┬─────┴──────┬──────┴─────┬──────┴─────┬──────┘
             ▼            ▼            ▼            ▼
      ┌───────────────────────────────────────────────────────┐
      │         Supervisor / Lecturer Analytics Layer         │
      │  Contribution % · Attendance · Burndown · PDF export  │
      └──────────────────────────┬────────────────────────────┘
                                 ▼
                 ┌─────────────────────────┐
                 │   Submission Module     │
                 │  GitHub · Drive · OneDrive│
                 │  Dropbox · Email · LMS  │
```

```
Cutting across every layer above:
─ Security: RBAC, audit log, encryption, rate limiting, CSP/HSTS
─ Account lifecycle: deletion, anonymization, identity reservation
─ Notifications: email + in-app, due-date and meeting reminders
```

How it fits together in practice:

1. A student registers either as an **individual** or as part of a **group workspace** (a group leader creates the workspace and issues join codes).
2. The **dashboard** is the hub — it surfaces tasks, deadlines, meetings, and notifications pulled from every module below it, and the new **command palette** lets a user jump to any of them without hunting through menus.
3. Inside a workspace, **tasks, meetings, budget, and documents** are the day-to-day working modules. Everything a member does here (task completion, attendance, contribution) feeds the **analytics layer**.
4. **Supervisors** sit above workspaces, not inside them — they view aggregated analytics and generated PDF reports across every group they oversee, without needing edit access to the underlying work.
5. The **AI Assistant** is an optional layer available from any module (meeting summaries, task generation, document summarization) and is fully gated by a kill switch and quota system so it can be turned off centrally without touching the modules that call it.
6. When a project is complete, the **submission module** packages the work out to GitHub, cloud storage, or an institution's LMS.
7. **Security and account lifecycle** are not a separate feature — they wrap every layer: identity is never exposed carelessly, actions are role-checked, and account deletion is safe by construction (can't delete a solely-owned active group's leader account, can't immediately reuse a deleted email).

---

5. Current Status

- Full production build passes cleanly (lint, type-check, and all routes compiling).
- Core phases 1 through 10 from the original roadmap (Foundation → Testing/Security/Deployment) are complete, with this session's work forming an additional hardening and polish pass on top of that baseline.
- AI features are intentionally disabled pending the user's decision on when to re-enable them.
- Outstanding, not-yet-started items: applying the new login page's visual treatment to the Register and Forgot Password pages, and extending the "primary actions + More menu" pattern to the department and supervisor pages.

This document was generated as a project retrospective and does not replace the technical documentation set (SRS, Development Journal, Configuration Guide, User Manual, *Maintenance Manual) already produced for EngiSync.*